

Vendor Invoice Intelligence & Payment Monitoring System

1. Project Overview

The Vendor Invoice Intelligence & Payment Monitoring System is a Python desktop application that helps organizations record vendor invoices, monitor payments, calculate outstanding balances, and identify overdue bills.

Organizations often receive invoices from several vendors, each with different payment terms, currencies, and due dates. Maintaining these records manually can cause duplicate entries, missed deadlines, inconsistent calculations, and difficulty determining how much is owed to each vendor.

This project brings invoice entry, validation, payment tracking, currency conversion, and reporting into one application. It uses a Tkinter graphical interface and a local SQLite database, allowing users to manage records without installing a database server.

The application uses rule-based validation and calculations. Its “intelligence” refers to automated checks, status classification, overdue detection, and financial summaries; it does not use a trained machine-learning model.

2. Business Problem and Vendor Challenges

Vendors depend on timely payments to maintain cash flow, purchase materials, pay employees, and continue supplying goods or services. Delayed or unclear payment information makes planning difficult.

The project addresses several common challenges:

Challenge	How the application helps
Invoices scattered across spreadsheets or files	Stores imported and manually entered invoices in one database
Duplicate invoice entries	Checks unique invoice IDs and vendor–invoice-number combinations
Uncertainty about payment progress	Shows invoice totals, recorded payments, and remaining balances
Partial payments that are difficult to track	Stores individual payments and recalculates the outstanding amount
Missed payment deadlines	Identifies overdue balances and displays alerts while the application is open
Invoices in different currencies	Preserves original amounts and provides conversion into a selected reporting currency
Difficulty reviewing vendor liabilities	Generates summaries grouped by vendor and original currency
Incorrect or incomplete input	Validates required details, dates, amounts, and payment consistency

The application supports the organization responsible for paying vendors. It improves visibility into payment obligations, but it does not transfer money or provide a vendor-facing portal.

3. Project Objectives

The main objectives are to:

- Provide a simple interface for recording and reviewing vendor invoices.
- Import invoice datasets from CSV and Excel files.
- Validate records before saving them.
- Prevent duplicate invoice entries.
- Track full and partial payments.
- Calculate outstanding balances accurately.
- Identify invoices with overdue unpaid amounts.
- Support reporting across multiple currencies.
- Generate invoice reports and vendor summaries.
- Demonstrate modular Python programming, object-oriented design, database handling, exception handling, and automated testing.

4. Main Functional Modules

4.1 Overview Dashboard

The Overview tab presents a summary of invoice and payment information for the selected reporting date.

It helps users review outstanding obligations, vendor-level balances, and overdue amounts. A reporting currency selector defaults to **INR**, and users can change it to another supported currency.

Original-currency totals remain separate so amounts in different currencies are not incorrectly added together.

4.2 Manual Invoice Entry

The Add invoice tab allows users to enter details from an actual invoice.

The form includes:

- Invoice ID and invoice number.
- Supplier ID, name, and tax ID.
- Invoice date, submission date, and due date.
- Invoice currency and total amount.
- Initial payment amount and payment date.
- Payment method and reference.
- Invoice state.

The application validates the information before saving it. For example, an invoice cannot have a due date earlier than its issue date, and a recorded payment cannot exceed the invoice amount.

4.3 CSV and Excel Import

Users can import .csv and .xlsx files instead of entering every invoice manually.

Each row represents one invoice. The required columns are:

invoice_id
supplier_id
supplier_name
invoice_number
invoice_date
invoice_due_date
invoice_amount
currency

Optional columns provide payment details, supplier tax information, submission dates, and source status information.

For Excel files, the importer reads the **first worksheet**, with column names in the first row. Formula cells are rejected; users must paste their calculated results as values before importing.

Each invoice is processed separately. Valid records are saved, while rejected records and warnings are reported with their reasons. A failed row does not undo other successfully imported invoices.

4.4 Invoice Validation and Duplicate Detection

The application checks that:

- Required fields contain values.
- Dates are valid.
- Invoice and submission dates are not in the future.
- Due and submission dates do not precede the invoice date.
- Amounts are valid, finite, and nonnegative.
- Active invoices have a positive total.
- Payments do not exceed the invoice amount.
- Positive payments have a valid payment date.
- Supplier details match existing records for the same supplier ID.
- Invoice IDs are unique.
- A supplier does not have the same invoice number entered twice.

These checks reduce data-entry errors and protect the accuracy of reports.

4.5 Payment Tracking

Users can record additional payments against an existing invoice.

Each payment stores its amount, date, method, and reference. Payments must use the invoice's original currency and cannot exceed its remaining balance.

The basic calculation is:

Outstanding balance = Invoice amount – Total recorded payments

For example:

Invoice amount: INR 10,000
First payment: INR 3,000
Second payment: INR 2,000
Outstanding balance: INR 5,000

For historical reporting, only payments dated on or before the selected reporting date are included.

4.6 Payment Status and Overdue Monitoring

The application determines payment status from the invoice and payment records:

Status	Meaning
Unpaid	No payment has been recorded
Partially paid	Some payment has been recorded, but a balance remains
Paid	Recorded payments equal the invoice amount
Cancelled	The invoice is marked cancelled and excluded from financial summaries

Overdue is an additional condition:

Outstanding balance > 0

AND

Reporting date > Due date

An invoice can therefore be **partially paid and overdue** at the same time. It becomes overdue after its due date has passed, rather than on the due date itself.

The application displays overdue alerts while it is running. Alerts use today's date even when a historical date is selected for reporting. It does not send email or SMS notifications or monitor invoices after the application closes.

4.7 Exchange Rates and Currency Conversion

The application supports:

INR, USD, EUR, GBP, CNY, SEK, BRL, CAD, and JPY.

INR is the default reporting currency. Users can select another supported currency without changing the original invoice amounts.

Exchange rates can be loaded from the online reference-rate provider or entered manually. Each stored rate has an effective date and source.

Rates are stored as **INR per one unit of foreign currency**. Cross-currency conversion uses:

Converted amount =

$$\text{Original amount} \times \text{INR per source currency unit} \\ \div \text{INR per target currency unit}$$

For example, using illustrative rates:

Outstanding amount: USD 100

USD rate: INR 80 per USD

EUR rate: INR 100 per EUR

$$\begin{aligned}\text{EUR equivalent} &= 100 \times 80 \div 100 \\ &= \text{EUR } 80\end{aligned}$$

Reports select the latest applicable rate on or before the reporting date. Missing rates are shown explicitly. These conversions provide reporting equivalents; they do not calculate bank charges or foreign-exchange settlement gains and losses.

4.8 Reports and Exports

The application provides:

- Invoice reports with balances, payment statuses, and overdue information.
- Vendor summaries grouped by supplier and original currency.
- Converted outstanding amounts in the selected reporting currency.
- Import issue reports identifying rejected records and warnings.

Reports can be exported as CSV files for review or further analysis. Table headings can be clicked to switch between ascending and descending sorting.

4.9 Record Correction and Database Clearing

Saved records are read-only. To correct an invoice, users delete the incorrect record and enter its replacement.

Deleting an invoice also deletes its linked payments. Individual payment records can also be deleted, which causes the invoice balance to be recalculated.

A separate database-clear option requires the exact confirmation password:

Delete

This is a guard against accidental deletion, not secure user authentication. Database clearing removes saved business records and rate history, after which default rates are restored.

5. Data Storage and Calculation Accuracy

The application uses **SQLite** for persistent local storage. Its main tables contain:

- Suppliers.
- Invoices.
- Payments.

- Exchange-rate history.
- Audit events.

Relationships connect invoices to suppliers and payments to invoices. Database constraints help prevent duplicates and invalid relationships.

Financial calculations use Python's **Decimal** type rather than binary floating-point arithmetic. Amounts are stored as exact decimal text in scaled units, supporting fractional inputs without silently rounding them during storage.

Converted reporting amounts are rounded to two decimal places.

6. Object-Oriented Design and Python Data Structures

The project separates responsibilities across classes and modules.

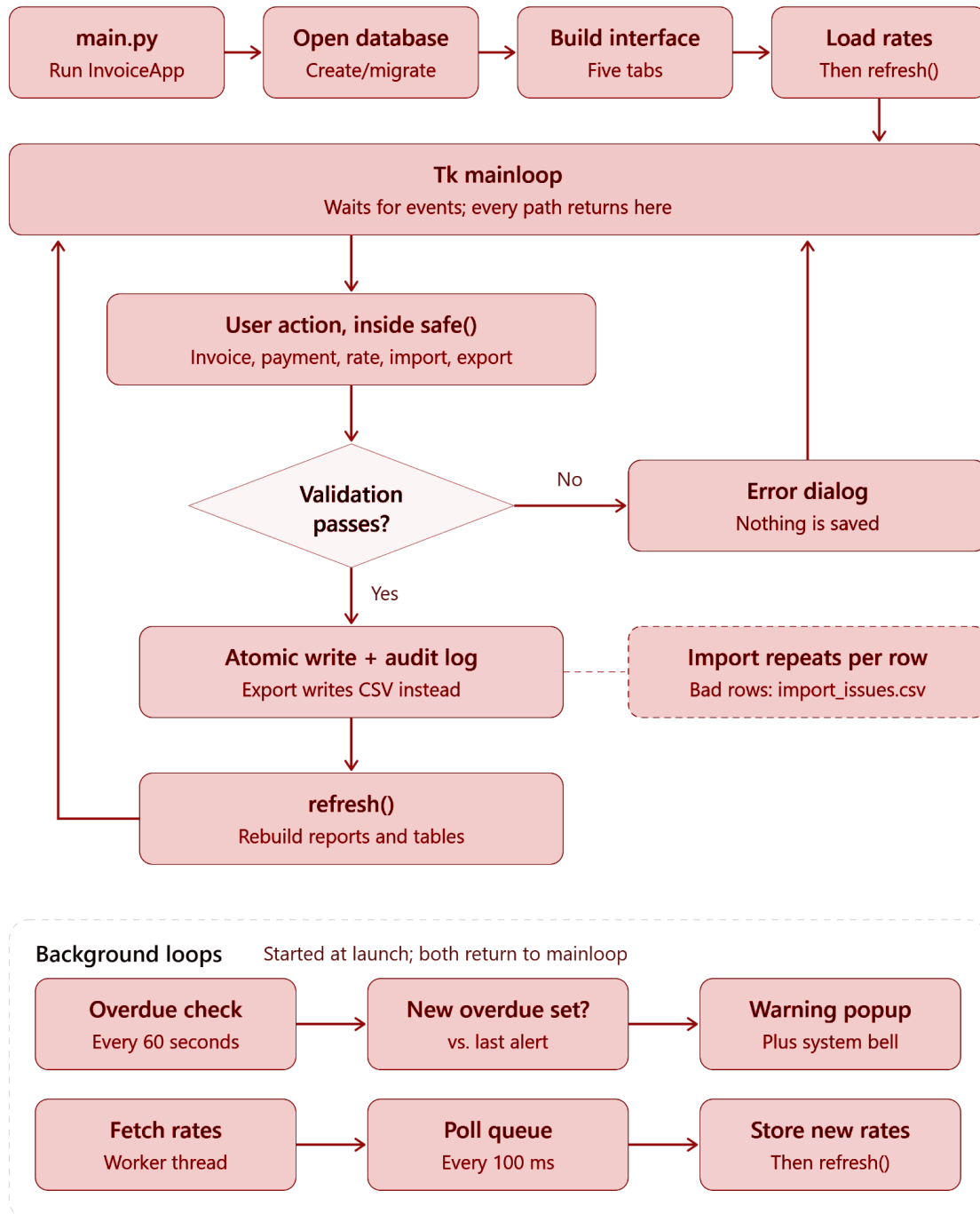
Component	Responsibility
Invoice	Represents invoice information and owns balance, status, and overdue behavior
Payment	Defines a payment value structure
InvoiceRepository	Handles SQLite persistence and retrieval
InvoiceService	Applies validation and invoice/payment business rules
CurrencyConverter	Manages dated rates and currency conversion
ReportGenerator	Produces invoice views and vendor summaries
InvoiceApp	Builds and coordinates the Tkinter interface

The design demonstrates:

- **Encapsulation:** Related data and behavior are grouped within classes.
- **Composition:** Services, repositories, converters, and reports work together through object references.
- **Inheritance:** The application class extends Tkinter's Tk class.
- **Modularity:** UI, validation, storage, reporting, and calculations are separated.

Lists hold collections of records, dictionaries map fields and accumulate totals, sets check required columns, and tuples represent grouping keys. Date operations use datetime, while a queue passes online-rate results from a background worker to the interface.

7. Functional Workflow



8. Testing and Verification

The project includes **43 automated pytest tests** covering important business rules and failure scenarios.

Tests cover validation, duplicates, partial payments, historical balances, exact decimal calculations, overpayment prevention, deletion, currency conversion, import handling, export behavior, sorting, database clearing, and database migration.

Tests use isolated temporary databases rather than modifying operational invoice records. The README also contains sample CSV inputs verified against the importer.

9. Technology Stack

Technology	Purpose
Python	Application logic and module structure
Tkinter	Desktop graphical interface
SQLite / sqlite3	Local database storage
OpenPyXL	Excel .xlsx import
csv	CSV import and export
decimal	Exact financial calculations
datetime	Date validation and overdue calculations
urllib.request	Fetching published exchange rates
logging	Operational event logging
pytest	Automated testing